课堂小测2

4. Consider a swapping system in which memory consists of the following hole sizes in memory order: 10 MB, 4 MB, 20 MB, 18 MB, 7 MB, 9 MB, 12 MB, and 15 MB. Which hole is taken for successive segment requests of

(a) 12 MB
(b) 10 MB
(c) 9 MB

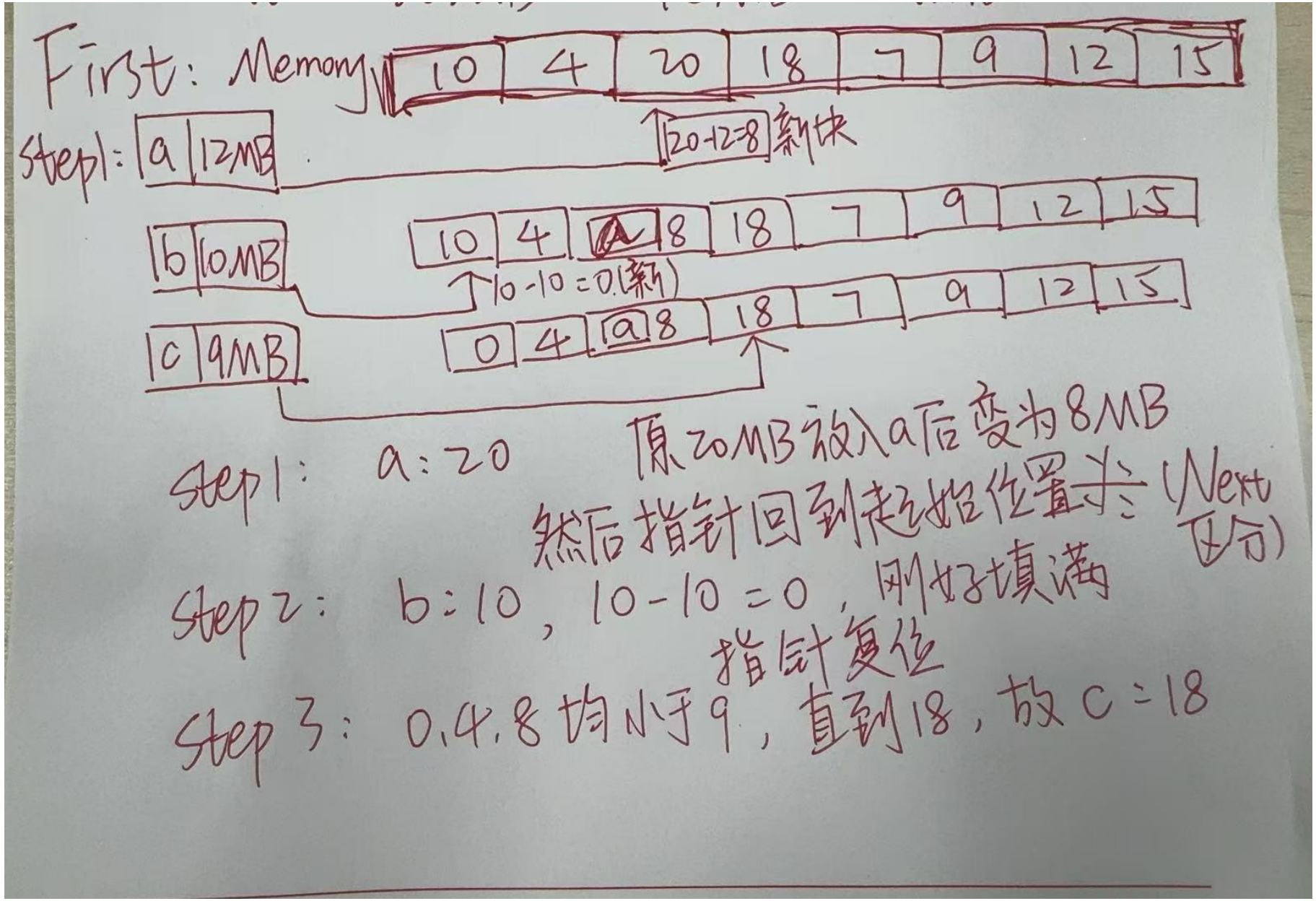
Answer：

	(a)	(b)	(c)
first	20MB	10MB	18MB
best	12MB	10MB	9MB
worst	20MB	18MB	15MB
next	20MB	18MB	9MB

课堂小测2(4)

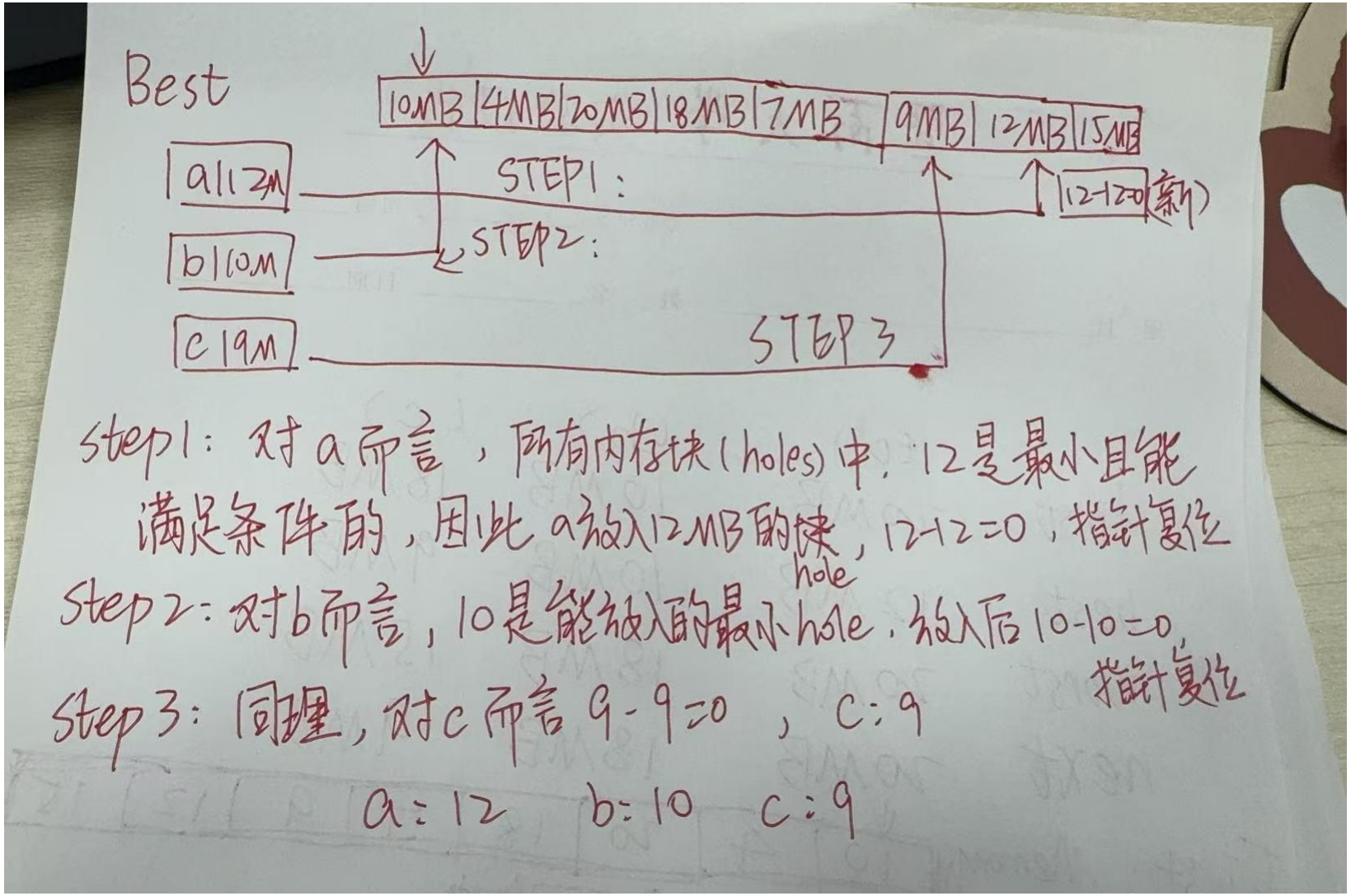
First:

遇到第一个满足条件的（能放下），放入后指针回到起始位置



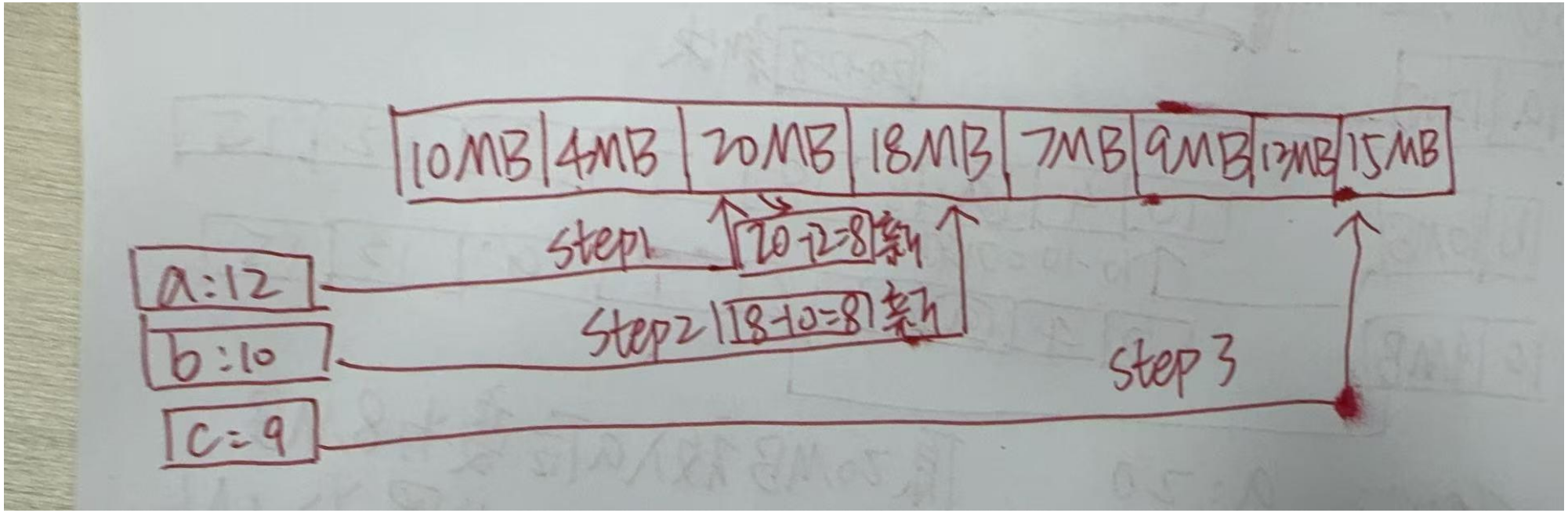
课堂小测2(4)

Best: 找到能放下且最小的，放入后指针回到起始位置



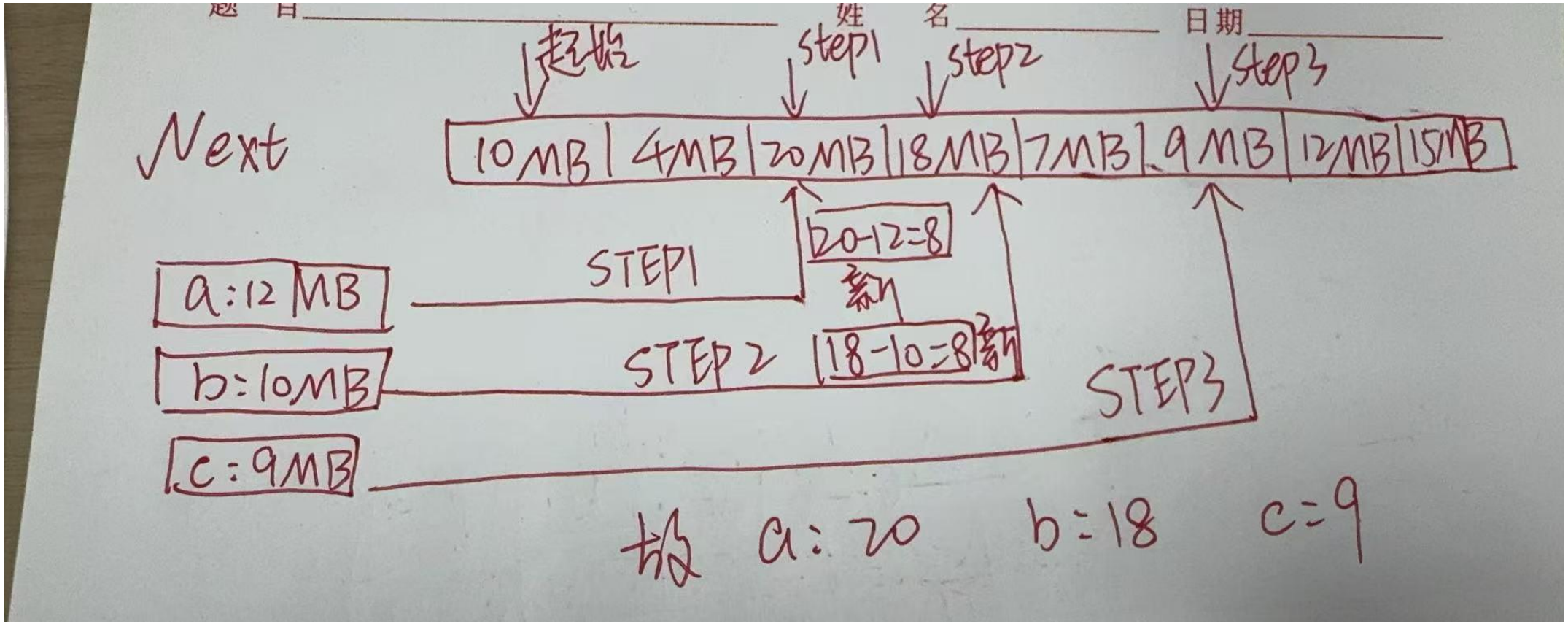
课堂小测2(4)

Worst: 找到能放下且最大的，放入后指针回到起始位置 （目的：避免外部碎片）



课堂小测2(4)

Next: 找到当前位置起第一个能放下的，放入后指针继续向后遍历



存储放置策略 Storage Placement Strategies

How to satisfy a request of size n from a list of free holes.从空闲孔列表中满足大小为 n 的请求。

❖ First Fit（首次适应算法）

思想：使用第一个可用的孔，其尺寸足以满足需要。Use the first available hole whose size is sufficient to meet the need.

Problem:产生平均大小的孔。Creates average size holes.

❖ Next Fit（）

思想：First Fit 的变形，从上次分配的地方（First fit 是从头开始找）开始找搜索最后一个孔停止，找到终点没有再转回去。Minor variation of first fit: search for the last hole stopped.

Problem: slightly worse performance than first fit.性能比 first fit 稍差。

❖Best Fit（最佳适应算法）

Use the hole whose size is equal to the need, or if none is equal, the hole that is larger but closest in size.使用大小与需要相等的孔，如果没有相同的孔，则使用较大但尺寸最接近的孔。

Problem: Creates small holes that can't be used.产生无法使用的小孔。

❖ Worst Fit（最坏适应算法）

Use the largest available hole.使用最大的可用孔。

Problem: Gets rid of large holes making it difficult to run large programs.去掉大孔会使运行大型程序变得困难。

课堂小测2（5）

5. What is the difference between a physical address and a virtual address?

考试中回答至少3个方面的不同点，规范分点

Feature	**Physical Address**	**Virtual Address**
Refers to	The actual, hardware-level location in RAM .	A logical address in the process's Virtual Memory space.
Generated by	The Memory Management Unit (MMU) after translation.	The CPU during a program's execution.
Visibility	Hidden from application programs.	Used directly by application programs.
Space Size	Limited by the **size** of the physical RAM chips.	**Can be much larger** than physical memory, using disk storage (swap space).
Purpose	To uniquely identify a location in the physical hardware.	To provide memory protection, abstraction, and allow for multitasking.

Chapter 2

30. Consider the following solution to the mutual-exclusion problem involving two processes *P0* and *P1*. Assume that the variable turn is initialized to 0. Process *P0*'s code is presented below.

```
/* Other code */

while (turn != 0) { } /* Do nothing and wait. */
Critical Section /* . . . */
turn = 0;

/* Other code */
```

For process *P1*, replace 0 by 1 in above code. Determine if the solution meets *all* the required conditions for a correct mutual-exclusion solution.

3 Conditions:

- **Mutual Exclusion (√)**
- **Progress (x)**
- **Bound Waiting (x)**

Solution: The solution satisfies mutual exclusion since it is not possible for both processes to be in their critical section. That is, when turn is 0, *P0* can execute its critical section, but not *P1*. Likewise, when turn is 1. However, this assumes *P0* must run first. If *P1* produces something and it puts it in a buffer, then while *P0* can get into its critical section, it will find the buffer empty and block. Also, this solution requires strict alternation of the two processes, which is undesirable.

Chapter 2

32. Show how counting semaphores (i.e., semaphores that can hold an arbitrary value) can be implemented using only binary semaphores and ordinary machine instructions.

信号量与P，V操作，书上有

Solution：

Associated with each counting semaphore are two binary semaphores, M, used for mutual exclusion, and B, used for blocking. Also associated with each counting semaphore is a counter that holds the number of ups minus the number of downs, and a list of processes blocked on that semaphore.

To implement down, a process first gains exclusive access to the semaphores, counter, and list by doing a down on M. It then decrements the counter. If it is zero or more, it just does an up on M and exits. If M is negative, the process is put on the list of blocked processes. Then an up is done on M and a down is done on B to block the process.

To implement up, first M is downed to get mutual exclusion, and then the counter is incremented. If it is more than zero, no one was blocked, so all that needs to be done is to up M. If, however, the counter is now neg ative or zero, some process must be removed from the list. Finally,an up is done on B and M in that order.

Chapter 2

36. A fast-food restaurant has four kinds of employees: (1) order takers, who take customers' orders; (2) cooks, who prepare the food; (3) packaging specialists, who stuff the food into bags; and (4) cashiers, who give the bags to customers and take their money. Each employee can be regarded as a communicating sequential process. What form of interprocess communication do they use? Relate this model to processes in UNIX.

Solution:

The employees communicate by passing messages: orders, food, and bags in this case,cashier. In UNIX terms, the four processes are connected by pipes

可以看作：

每个员工是一个独立的UNIX进程。

连接员工的“沟通渠道”就是UNIX管道。

整个工作流程可以看作一个由管道连接的进程链：

Chapter 2

37. Suppose that we have a message-passing system using mailboxes. When sending to a full mailbox or trying to receive from an empty one, a process does not block. Instead, it gets an error code back. The process responds to the error code by just trying again, over and over, until it succeeds. Does this scheme lead to race conditions?

Solution:
It does not lead to race conditions (nothing is ever lost), but it is effectively busy waiting.

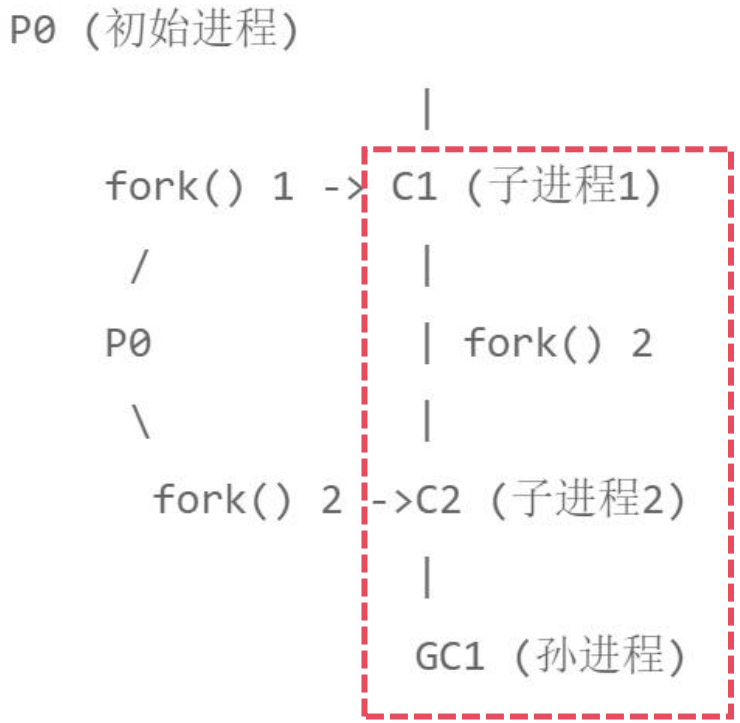
Race： Inconsistent or not?
page67

Chapter 2

39. Consider the following piece of C code:

```
void main( ) {
    fork( );
    fork( );
    exit( );
}
```

How many child processes are created upon execution of this program?



Solution：

Three processes are created. After the initial process forks, there are two processes running, a parent and a child. Each of them then forks, creating two additional processes. Then all the processes exit.

Chapter 2

43. Measurements of a certain system have shown that the average process runs for a time T before blocking on I/O. A process switch requires a time S, which is effectively wasted (overhead). For round-robin scheduling with quantum Q, give a formula for the CPU efficiency for each of the following:

(a) $Q = \infty$
(b) $Q > T$
(c) $S < Q < T$
(d) $Q = S$
(e) Q nearly 0

Solution:

43. The CPU efficiency is the useful CPU time divided by the total CPU time. When $Q \geq T$, the basic cycle is for the process to run for T and undergo a process switch for S. Thus, (a) and (b) have an efficiency of $T/(S + T)$. When the quantum is shorter than T, each run of T will require T/Q process switches, wasting a time ST/Q. The efficiency here is then

$$\frac{T}{T + ST/Q}$$

which reduces to $Q/(Q + S)$, which is the answer to (c). For (d), we just substitute Q for S and find that the efficiency is 50%. Finally, for (e), as $Q \rightarrow 0$ the efficiency goes to 0.

Chapter 2

43. Measurements of a certain system have shown that the average process runs for a time T before blocking on I/O. A process switch requires a time S, which is effectively wasted (overhead). For round-robin scheduling with quantum Q, give a formula for the CPU efficiency for each of the following:

(a) $Q = \infty$
(b) $Q > T$
(c) $S < Q < T$
(d) $Q = S$
(e) Q nearly 0

(a) Q = ∞ 和 (b) Q > T

这两种情况的本质是相同的。当时间片 Q无限大 (∞) 或大于进程的平均运行时间 T时，进程在**一次时间片内**就能完成其本次的运行，直到因I/O而主动阻塞。因此，在它阻塞之前，不会被调度器因为时间片用完而强制中断。

$$\text{Efficiency} = \frac{T}{T + S}$$

(c) S < Q < T

此时时间片 Q小于进程的平均运行时间 T，**进程还来不及阻塞，时间片就用完了**，因此会被调度器强制中断。要运行 T时间，每个周期运行 Q时间，所以需要 T / Q个周期。有效时间仍然是 T。每个周期结束后都有一次进程切换开销 S。总共有 T/Q个周期，所以总浪费时间为 (T/Q) * S。总时间：T + (T/Q) * S = T + ST/Q

$$\text{Efficiency} = \frac{T}{T + \frac{ST}{Q}} = \frac{1}{1 + \frac{S}{Q}} = \frac{Q}{Q + S}$$

(d) Q = S

这是情况 (c) 的一个特例，可直接将 Q = S代入情况 (c) 的公式中。

$$\text{Efficiency} = \frac{Q}{Q + S} = \frac{S}{S + S} = \frac{S}{2S} = \frac{1}{2} = 50\%$$

(e) Q 几乎为 0

这是时间片极端的短的情况。进程每次只能运行一个极短的时间（几乎为0），然后就要进行一次耗时为 S的进程切换。有效时间趋近于 0，在极短的时间内，系统几乎全部在进行进程切换，浪费的时间 S占主导。

$$\lim_{Q\to 0} \text{Efficiency} = \lim_{Q\to 0} \frac{Q}{Q+S} = \frac{0}{0+S} = 0$$

Result of a-e：

(a) & (b)	Q ≥ T	$\frac{T}{T+S}$
(c)	S < Q < T	$\frac{Q}{Q+S}$
(d)	Q = S	50%
(e)	Q → 0	0

Chapter 6

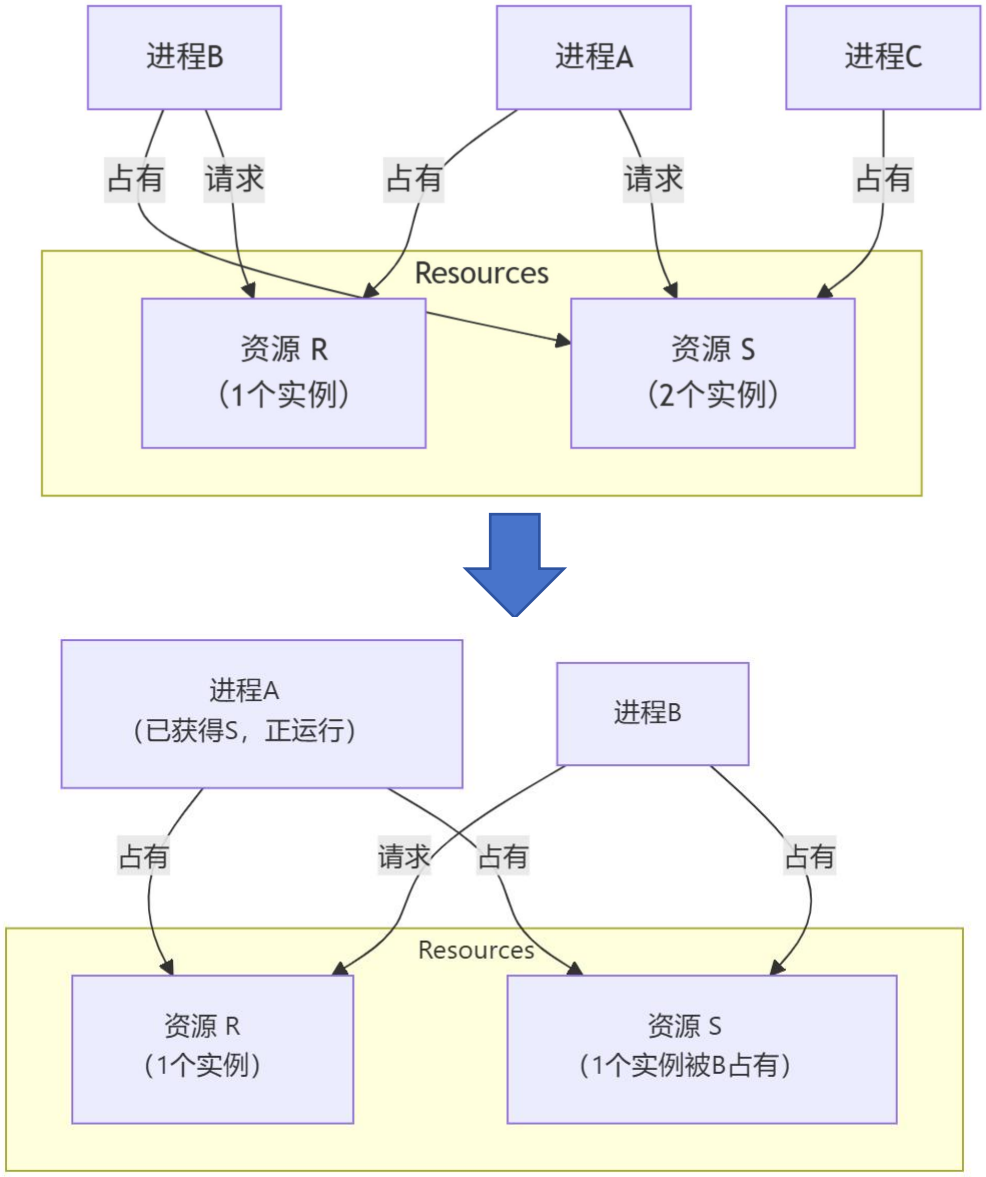
5. The four conditions (mutual exclusion, hold and wait, no preemption and circular wait) are necessary for a resource deadlock to occur. Give an example to show that these conditions are not sufficient for a resource deadlock to occur. When are these conditions sufficient for a resource deadock to occur?

Solution:

5. Suppose that there are three processes, A, B and C, and two resource types, R and S. Further assume that there are one instance of R and two instance of S. Consider the following execution scenario: A requests R and gets it; B requests S and gets; C requests S and gets it (there are two instances of S); B requests R and is blocked; A requests S and is blocked. At this stage all four conditions hold. However, there is no deadlock. When C finishes, one instance of S is released that is allocated to A. Now A can complete its execution and release R that can be allocated to B, which can then complete its execution. These four conditions are enough if there is one resource of each type.

These four conditions is NOT sufficient to guarantee a deadlock.

Chapter 6（5）



进程C不依赖于A或B所占有的任何资源，因此它可以独立地继续运行直至结束。当C完成后，它会释放其占有的资源S。

这个可用的实例S可以被分配给正在等待它的进程（即进程A）。

进程A的请求边得到满足。A现在占有着R和S，可以完成它的工作。

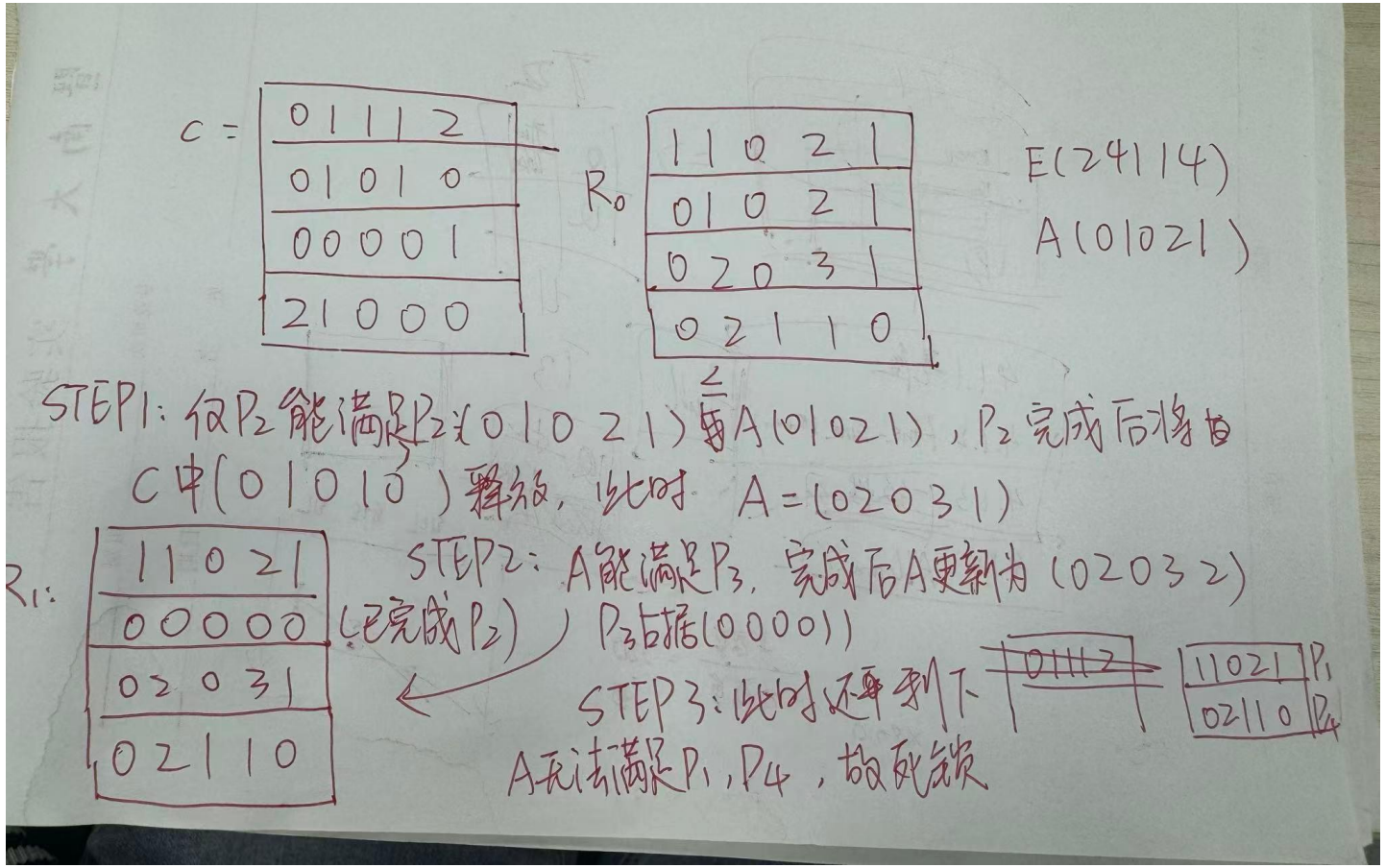
随后，A完成工作释放所有资源，B工作；

因此，不会死锁

Chapter 6

15. Explain how the system can recover from the deadlock in previous problem using

(a) recovery through preemption.
(b) recovery through rollback.
(c) recovery through killing processes.



14题：
在P2，P3完成后，P1与P4形成死锁

Chapter 6

15. Explain how the system can recover from the deadlock in previous problem using

(a) recovery through preemption.
(b) recovery through rollback.
(c) recovery through killing processes.

(a) Recovery through preemption:

After processes P2 and P3 complete, process P1 can be forced to preempt 1 unit of RS3. This will makeA=(0 2 1 3 2), and allow process P4 to complete. Once P4 completes and release its resources P1 may complete.

(b) Recovery through rollback:

Rollback P1 to the state checkpointed before it acquired RS3.

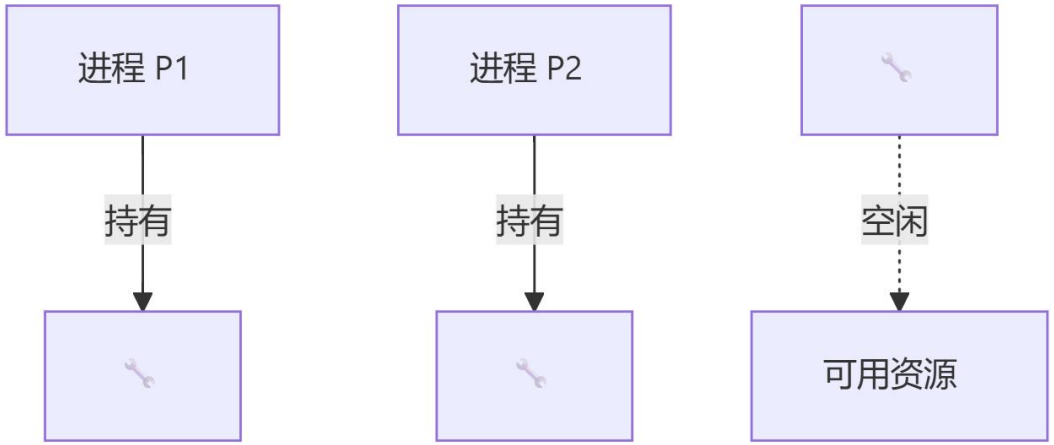
(c) Recovery through killing processes: Kill P1.

Chapter 6

22. A system has two processes and three identical resources. Each process needs a maximum of two resources. Is deadlock possible? Explain your answer.

Solution：

22. The system is deadlock free. Suppose that each process has one resource. There is one resource free. Either process can ask for it and get it, in which case it can finish and release both resources. Consequently, deadlock is impossible.

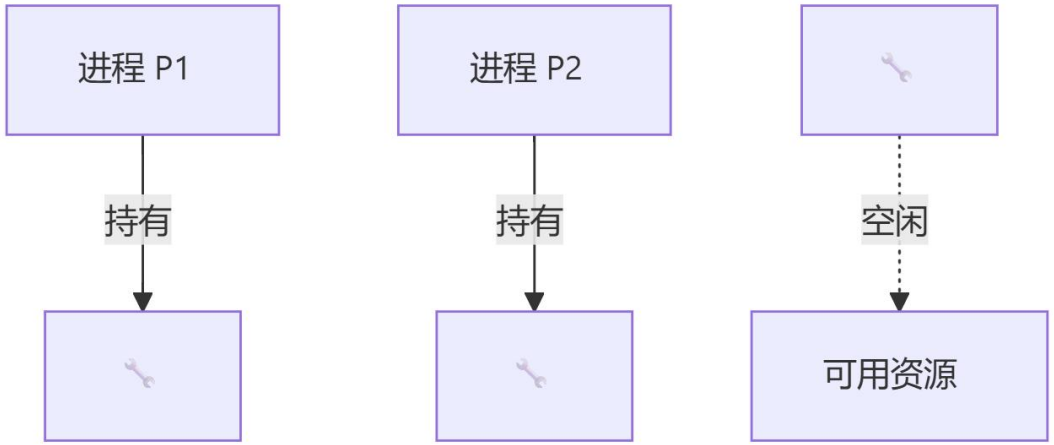


Chapter 6

22. A system has two processes and three identical resources. Each process needs a maximum of two resources. Is deadlock possible? Explain your answer.

Solution：

22. The system is deadlock free. Suppose that each process has one resource. There is one resource free. Either process can ask for it and get it, in which case it can finish and release both resources. Consequently, deadlock is impossible.



Chapter 6

23. Consider the previous problem again, but now with p processes each needing a maximum of m resources and a total of r resources available. What condition must hold to make the system deadlock free?

Solution：

23. If a process has m resources it can finish and cannot be involved in a deadlock. Therefore, the worst case is where every process has $m - 1$ resources and needs another one. If there is one resource left over, one process can finish and release all its resources, letting the rest finish too. Therefore the condition for avoiding deadlock is $r \geq p(m - 1) + 1$.

最坏情况（即能够发生死锁的极限条件）：

- 每个进程已经持有的资源数是 **m - 1。**
- p个进程总共已经持有的资源数是 **p * (m - 1)。**

此时，任何一个进程若是能再得到一个资源(总数=**p * (m - 1)+1**)，就不会死锁，
因此，r只要**不小于 p * (m - 1)+1** 就**不会**发生死锁

Chapter 6

25. The banker's algorithm is being run in a system with m resource classes and n processes. In the limit of large m and n, the number of operations that must be performed to check a state for safety is proportional to $m^a n^b$. What are the values of a and b?

Solution：

25. Comparing a row in the matrix to the vector of available resources takes m operations. This step must be repeated on the order of n times to find a process that can finish and be marked as done. Thus, marking a process as done takes on the order of mn steps. Repeating the algorithm for all n processes means that the number of steps is then mn^2. Thus, $a = 1$ and $b = 2$.

时间复杂度计算O（$m^a n^b$）

已知n个进程，m类资源

在一轮之中，银行家算法需要检查现有资源是否能满足进程所需的资源（单次比较一个进程需要O(m) 次操作），遍历n个进程的m个资源并与可分配资源比较，然后更新可用资源，因此是O(n*m)

总共n个进程，需要进行n轮

O（n*（n*m））=O（mn^2）

Chapter 6

26. A system has four processes and five allocatable resources. The current allocation and maximum needs are as follows:

	Allocated	*Maximum*	*Available*
Process A	1 0 2 1 1	1 1 2 1 3	0 0 x 1 1
Process B	2 0 1 1 0	2 2 2 1 0	
Process C	1 1 0 1 0	2 1 3 1 0	
Process D	1 1 1 1 0	1 1 2 2 1	

What is the smallest value of x for which this is a safe state?

Solution：

26. The needs matrix is as follows:

0 1 0 0 2
0 2 1 0 0
1 0 3 0 0
0 0 1 1 1

If x is 0, we have a deadlock immediately. If x is 1, process D can run to completion. When it is finished, the available vector is 1 1 2 2 1. Unfortunately we are now deadlocked. If x is 2, after D runs, the available vector is 1 1 3 2 1 and C can run. After it finishes and returns its resources the available vector is 2 2 3 3 1, which will allow B to run and complete, and then A to run and complete. Therefore, the smallest value of x that avoids a deadlock is 2.

1. 计算Need矩阵
2. 分析不同x值下的安全状态